API Documentation

Base URL

`http://localhost:3000`

User APIs

1. Signup

Endpoint:

`POST /api/users/signup`

Description: Registers a new user.

Request Body:

```
{
  "username": "string",
  "email": "string",
  "password": "string",
  "phone": "string"
}
```

Response: - **201 Created** `json { "message": "User registered successfully", "user": { "username": "string", "email": "string" }, "accessToken": "jwt-access-token", "refreshToken": "jwt-refresh-token" } - 400 Bad Request json { "error": "All fields are required" } - 400 Bad Request json { "error": "Email already exists" } - 500 Internal Server Error json { "error": "Internal server error" }`

---

2. Login

Endpoint:

`POST /api/users/login`

Description: Logs in a user with valid credentials.

Request Body:

```
{
  "email": "string",
  "password": "string"
}
```

Response: - **200 OK** `json { "message": "Login successful", "user": { "username": "string", "email": "string" }, "accessToken": "jwt-access-token", "refreshToken": "jwt-refresh-token" }` - **400 Bad Request** `json { "error": "Email and password are required" }` - **401 Unauthorized** `json { "error": "Invalid credentials" }` - **500 Internal Server Error** `json { "error": "Internal server error" }`

---

3. Refresh Token

Endpoint:

`POST /refresh`

Description: Exchanges a valid refresh token for a new access token and refresh token.

Request Body:

```
{
  "refreshToken": "jwt-refresh-token"
}
```

Response: - **200 OK** `json { "accessToken": "new-jwt-access-token", "refreshToken": "new-jwt-refresh-token" }` - **400 Bad Request** `json { "error": "Refresh token required" }` - **401 Unauthorized** `json { "error": "Refresh token expired" }` - **401 Unauthorized** `json { "error": "Invalid refresh token" }`

Contact Us (Ticket Creation)

Endpoint:

`POST /contact-us`

Description: Submits a contact request, stores it as a ticket in the database (with status 'open'), and sends an email to all admin emails.

Request Body:

```
{
  "name": "string",
  "email": "string",
  "message": "string"
}
```

Response: - **200 OK** `json { "success": true, "message": "Message sent successfully.", "sentTo": [ { "email":`

"admin1@example.com" }, { "email": "admin2@example.com" }], "ticket": { "_id": "string", "name": "string", "email": "string", "message": "string", "status": "open", "createdAt": "2025-07-12T03:27:14.100Z", "updatedAt": "2025-07-12T03:27:14.100Z" } } - **400 Bad Request** json { "error": "Name, email, and message are required." } - **500 Internal Server Error** json { "error": "Failed to save ticket." } or json { "error": "Failed to fetch admin emails." } or json { "error": "Failed to send message." }

Notes: - All errors are logged to `logs/error.log`. - Tickets are created with status `open` by default. - Admin emails are fetched from the database.

Category APIs

1. Create Category

Endpoint:

`POST /api/categories`

Description: Creates a new category.

Request Body:

```
{
  "categoryName": "string",
  "description": "string"
}
```

Response: - **201 Created** json { "message": "Category created successfully", "category": { "_id": "string", "categoryName": "string", "description": "string", "createdAt": "2025-07-12T03:27:14.100Z", "updatedAt": "2025-07-12T03:27:14.100Z" } } - **400 Bad Request** json { "error": "Category name and description are required" } - **409 Conflict** json { "error": "Category name already exists" } - **500 Internal Server Error** json { "error": "Internal server error" }

---

2. Get All Categories

Endpoint:

`GET /api/categories`

Description: Retrieves all categories.

Response: - **200 OK** `json { "categories": [ { "_id": "string", "categoryName": "string", "description": "string", "createdAt": "2025-07-12T03:27:14.100Z", "updatedAt": "2025-07-12T03:27:14.100Z" } ] }` - **500 Internal Server Error** `json { "error": "Internal server error" }`

---

3. Get Specific Category

Endpoint:

`GET /api/categories/{categoryId}`

Description: Retrieves a specific category by MongoDB ID.

Parameters: - `categoryId` (path parameter): The MongoDB ObjectId of the category

Response: - **200 OK** `json { "category": { "_id": "string", "categoryName": "string", "description": "string", "createdAt": "2025-07-12T03:27:14.100Z", "updatedAt": "2025-07-12T03:27:14.100Z" } }` - **404 Not Found** `json { "error": "Category not found" }` - **500 Internal Server Error** `json { "error": "Internal server error" }`

---

4. Update Category

Endpoint:

`PUT /api/categories/{categoryId}`

Description: Updates an existing category.

Parameters: - `categoryId` (path parameter): The MongoDB ObjectId of the category

Request Body:

```
{
  "categoryName": "string",
  "description": "string"
}
```

Response: - **200 OK** `json { "message": "Category updated successfully", "category": { "_id": "string", "categoryName": "string", "description": "string", "createdAt": "2025-07-12T03:27:14.100Z", "updatedAt": "2025-07-12T03:27:14.100Z" } }` - **404 Not Found** `json { "error": "Category not found" }` - **400 Bad Request** `json { "error": "Invalid input data" }`

- **409 Conflict** `json { "error": "Category name already exists" }` - **500 Internal Server Error** `json { "error": "Internal server error" }`

---

5. Delete Category

Endpoint:

`DELETE /api/categories/{categoryId}`

Description: Deletes a category.

Parameters: - `categoryId` (path parameter): The MongoDB ObjectId of the category

Response: - **200 OK** `json { "message": "Category deleted successfully" }` - **404 Not Found** `json { "error": "Category not found" }` - **500 Internal Server Error** `json { "error": "Internal server error" }`

Product APIs

1. Create Product

Endpoint:

`POST /api/products`

Description: Creates a new product. The product ID is automatically generated by MongoDB.

Request Body:

```
{
  "productName": "string",
  "category": "string",
  "price": 0,
  "pictures": ["string"],
  "stockQuantity": 0
}
```

Response: - **201 Created** `json { "message": "Product created successfully", "product": { "_id": "string", "productName": "string", "category": "string", "price": 0, "pictures": ["string"], "stockQuantity": 0, "createdAt": "2025-07-12T03:27:14.100Z", "updatedAt": "2025-07-12T03:27:14.100Z" } }` - **400 Bad Request** `json { "error": "Product name, category, and price are required" }` - **404 Not Found** `json { "error": "Category`

not found" } - **500 Internal Server Error** json { "error": "Internal server error" }

---

2. Get All Products

Endpoint:

GET /api/products

Description: Retrieves all products with populated category information.

Response: - **200 OK** json { "products": [{ "_id": "string", "productName": "string", "category": { "_id": "string", "categoryName": "string", "description": "string" }, "price": 0, "pictures": ["string"], "stockQuantity": 0, "createdAt": "2025-07-12T03:27:14.100Z", "updatedAt": "2025-07-12T03:27:14.100Z" }] } - **500 Internal Server Error** json { "error": "Internal server error" }

---

3. Get Specific Product

Endpoint:

GET /api/products/{productId}

Description: Retrieves a specific product by MongoDB ID with populated category information.

Parameters: - productId (path parameter): The MongoDB ObjectId of the product

Response: - **200 OK** json { "product": { "_id": "string", "productName": "string", "category": { "_id": "string", "categoryName": "string", "description": "string" }, "price": 0, "pictures": ["string"], "stockQuantity": 0, "createdAt": "2025-07-12T03:27:14.100Z", "updatedAt": "2025-07-12T03:27:14.100Z" } } - **404 Not Found** json { "error": "Product not found" } - **500 Internal Server Error** json { "error": "Internal server error" }

---

4. Update Product

Endpoint:

PUT /api/products/{productId}

Description: Updates an existing product.

Parameters: - `productId` (path parameter): The MongoDB ObjectId of the product

Request Body:

```
{
  "productName": "string",
  "category": "string",
  "price": 0,
  "pictures": ["string"],
  "stockQuantity": 0
}
```

Response: - **200 OK** `json { "message": "Product updated successfully", "product": { "_id": "string", "productName": "string", "category": { "_id": "string", "categoryName": "string", "description": "string" }, "price": 0, "pictures": ["string"], "stockQuantity": 0, "createdAt": "2025-07-12T03:27:14.100Z", "updatedAt": "2025-07-12T03:27:14.100Z" } }` - **404 Not Found** `json { "error": "Product not found" }` - **400 Bad Request** `json { "error": "Invalid input data" }` - **500 Internal Server Error** `json { "error": "Internal server error" }`

---

5. Delete Product

Endpoint:

`DELETE /api/products/{productId}`

Description: Deletes a product.

Parameters: - `productId` (path parameter): The MongoDB ObjectId of the product

Response: - **200 OK** `json { "message": "Product deleted successfully" }` - **404 Not Found** `json { "error": "Product not found" }` - **500 Internal Server Error** `json { "error": "Internal server error" }`